Práctica 3. Navegación con tile coding

(Sesión 1. Duración: 1 sesión)

PREPARACIÓN DEL ENTORNO DE PROGRAMACIÓN

La práctica se ejecuta con una instalación local de Python. Si ya tienes el entorno de las prácticas anteriores, te sirve tal cual.

1. **Instalar Python 3.10 o posterior**. Si no lo tienes, descárgalo de la página oficial de Python o utiliza una instalación de Anaconda. Esta sesión no usa `pygame`, así que también sirve la 3.14.

2. **Crear un entorno virtual** para la asignatura y activarlo. Desde la carpeta de la práctica:

   ```
   python -m venv .venv
   .venv\Scripts\activate        (Windows)
   source .venv/bin/activate     (macOS y Linux)
   ```

3. **Instalar las dependencias** en el entorno virtual:

   ```
   pip install numpy gymnasium matplotlib
   ```

4. **Comprobar que todo funciona**: con el entorno virtual activado, ejecuta desde el terminal (el valor de α no importa para esta comprobación)

   ```
   python main.py --episodios 1 --alpha 0
   ```

   Debe terminar con este mensaje, que avisa de que todavía no has programado los huecos:

   ```
   NotImplementedError: Implementa epsilon_sarsa y borra esta línea.
   ```

   Si en su lugar aparece `ModuleNotFoundError`, falta el paquete que nombra el error: instálalo con `pip install` en el mismo entorno y vuelve a ejecutar la instrucción.

1. Introducción

En esta sesión comenzamos a trabajar de cara a la presentación final de la asignatura. Para facilitar la comprensión del paso de aprendizaje tabular a función de aproximación, planteamos una práctica guiada en la que aprenderemos a modelar espacios continuos con una discretización que permite generalizar de forma razonadamente controlada la experiencia obtenida desde una posición a un determinado entorno de esta: el *tile coding*.

El funcionamiento de los métodos que permiten a un agente aprender con función de aproximación puede complicarse mucho. Por ello, con la idea de que puedas comprender con suficiente profundidad este proceso, lo vamos a simplificar notablemente en esta primera sesión. Por un lado, vamos a usar un entorno sencillo, en el que solo tenemos que llegar a una zona objetivo desde una posición de partida aleatoria evitando chocarnos con los obstáculos (paredes). En la Figura 1 se observa un posible estado del entorno. El agente se representa en color naranja y la zona objetivo en verde.

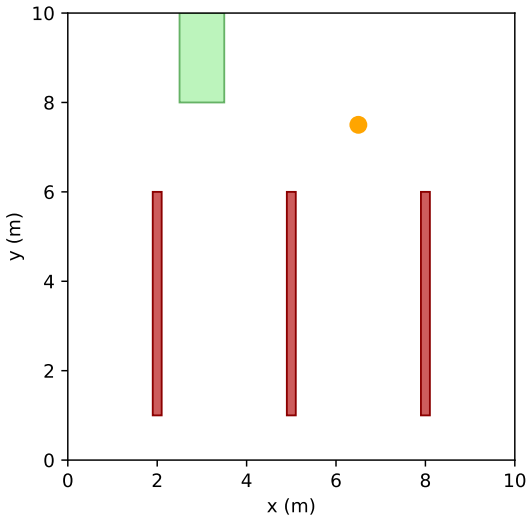


Figura 1. El entorno de navegación. Los obstáculos están en rojo, la zona objetivo en verde y el agente, en naranja.

Por otro lado, en parte gracias a la sencillez del entorno, vamos a usar aproximación lineal de la función de valor de acción con *tile coding*, y vamos a aprenderla con dos algoritmos: semi-gradient Sarsa y semi-gradient Q-learning.

ENTORNO

- El entorno tiene forma cuadrada, con 10 metros de lado. El área objetivo es un rectángulo cuyo vértice inferior izquierdo se encuentra en $(x = 2{,}5,\ y = 8)$, con dimensiones $(l_x = 1,\ l_y = 2)$. Hay tres obstáculos con dimensiones $(l_x = 0{,}2,\ l_y = 5)$, cuyos vértices inferiores izquierdos se encuentran en $(x = 1{,}9,\ y = 1)$, $(x = 4{,}9,\ y = 1)$ y $(x = 7{,}9,\ y = 1)$.
- El agente es un círculo de $0{,}2$ m de radio. En todos los estados puede elegir una acción de $\mathcal{A}(s) = \{\textit{arriba}, \textit{abajo}, \textit{izquierda}, \textit{derecha}\}$, numeradas de 0 a 3 en ese orden, y se desplaza $0{,}25$ m en la dirección elegida.
- La interacción es episódica. El episodio termina (`terminated` a `True`) tanto si el agente llega a la zona objetivo (éxito: su centro entra en ella) como si choca con alguna de las paredes del perímetro o con un obstáculo (fracaso: el círculo los toca). Por lo tanto, podemos modelar la tarea como un MDP episódico. Además, el episodio se trunca (`truncated` a `True`) si llega a 2000 pasos sin terminar.
- Cada episodio empieza en una posición aleatoria, uniforme entre las posiciones libres del recinto.
- Tras cada acción, el entorno devuelve la observación «cruda» $(x, y, \textit{colisión}, \textit{objetivo})$: la nueva posición del agente y dos variables binarias que indican si ha habido colisión o si se ha llegado a la zona objetivo. Es importante notar que esta no es la señal que llega al agente: el agente recibe la codificación de la posición con *tile coding* (Sección 3).
- La recompensa de cada paso la diseñas tú (Sección 3).

AGENTE

- El agente aproxima el valor de acción con una función lineal de las características de *tile coding*, con un vector de pesos para cada acción:

$$\hat{q}(s, a, \mathbf{w}) = \mathbf{w}_a^\top \mathbf{x}(s),$$

  donde $\mathbf{x}(s)$ es el vector binario que codifica la posición del agente.

- Se entrena con semi-gradient Sarsa o con semi-gradient Q-learning y una política ϵ-greedy. Se evalúa con su política greedy ($\epsilon = 0$), que es determinista.

2. Trabajo previo

- Lee la sección 9.5.4 de Sutton y Barto, *Reinforcement Learning: An Introduction* (2.ª ed., pp. 217-221), donde se presenta el concepto de *tile coding*.

- Analiza el área en la que influye una muestra de experiencia. Cuando el agente actualiza sus pesos en una posición, cambia el valor estimado de todas las posiciones que comparten con ella algún *tile*. El script `influencia.py` dibuja, para un punto del recinto, los *tiles* activos de cada *tiling* y la fracción de *tiles* que comparte con él cada punto del recinto:

```
python influencia.py --n_tiles 10 10 --num_tilings 8 --punto 5 7.5
```

  Prueba varios valores del número de *tiles* por dimensión (`--n_tiles`) y del número de *tilings* (`--num_tilings`), y analiza cómo cambian el tamaño y la forma de esa área. Compara también los desplazamientos uniformes y asimétricos entre *tilings* (`--desplazamiento uniforme`; Figura 9.11 del libro). Relaciónalo con el equilibrio entre la especificidad y la generalización de la experiencia.

- Analiza el pseudocódigo del algoritmo semi-gradient Sarsa (Sutton y Barto, p. 244) y piensa qué cambia en semi-gradient Q-learning.

- Antes de comenzar la sesión habrá un test evaluable con preguntas sobre este trabajo previo y, en general, sobre la aproximación lineal de la función de valor. Del trabajo previo no hay que entregar nada.

3. Representación del estado y diseño de la recompensa

REPRESENTACIÓN DEL ESTADO

El script `representacion.py` contiene la clase `TileCoder`, que hace de pasarela entre el entorno y el agente: convierte la posición (x, y) en los índices de las componentes de $\mathbf{x}(s)$ que valen 1, una por *tiling*. No hay que programarla ni modificarla. Los *tilings* se desplazan entre sí según el vector $(1, 3)$ que recomienda el libro, y su diseño lo eliges tú desde la línea de comandos: `--n_tiles` fija el número de *tiles* por dimensión de cada *tiling* y `--num_tilings`, el número de *tilings*. Con n *tilings* de $n_x \times n_y$ *tiles*, $\mathbf{x}(s)$ tiene $n\,(n_x + 1)(n_y + 1)$ componentes, porque al desplazarse cada *tiling* necesita un *tile* más por dimensión para cubrir el recinto.

DISEÑO DE LA RECOMPENSA

Para seguir el estándar de *gymnasium*, la recompensa se calcula dentro de la función `step(.)` del entorno, que llama a la función `recompensa` de `recompensa.py`. Prográmala dentro del espacio marcado: recibe la posición antes y después del movimiento y las dos variables binarias de colisión y llegada. Lo que decidas aquí define la tarea que el agente aprende de verdad, así que justifica en tu memoria de trabajo por qué, con tu recompensa, maximizar el retorno lleva a llegar a la zona objetivo sin chocar y por el camino más corto.

4. Entrenamiento del agente

La carpeta de la práctica contiene los siguientes ficheros. Los huecos que hay que programar están marcados con `START CODE HERE ... END CODE HERE`, y no se debe modificar nada fuera de ellos:

- `main.py`: es el script principal. Entrena, evalúa y guarda el agente. Todos los hiperparámetros se pasan por la línea de comandos, así que **no hay que editarlo** (`python main.py --help` los lista todos):

```
python main.py --algoritmo <sarsa/q_learning/ambos> --alpha <alpha>
               --n_tiles 10 10 --num_tilings 8 --graficas --mapa
```

- `env.py`, `representacion.py`, `entrenamiento.py` y `evaluacion.py`: el entorno, el *tile coding*, el bucle de entrenamiento (repite episodios y registra los datos de las gráficas) y la evaluación de la política greedy. No tienen huecos y no hay que modificarlos.
- `agente.py`: la clase `AgenteLineal`, que guarda los pesos $\mathbf{w}$, y dos huecos: `valores_q`, que calcula $\hat{q}(s, a, \mathbf{w})$ para todas las acciones a partir de los *tiles* activos, y `epsilon_greedy`.
- `sarsa.py`: dos huecos. `episodio_sarsa` ejecuta un episodio completo de semi-gradient Sarsa, y `epsilon_sarsa` da el valor de ϵ de cada episodio del entrenamiento.
- `q_learning.py`: lo mismo para semi-gradient Q-learning, en los huecos `episodio_q_learning` y `epsilon_q_learning`.
- `recompensa.py`: el hueco `recompensa` (Sección 3).
- `influencia.py` y `evalua_agente.py`: la herramienta del trabajo previo y la comprobación de los agentes que se envían a la clasificación (Sección 5).

El tamaño de paso α (`--alpha`) no tiene valor por defecto: elígelo tú. El equilibrio entre exploración y explotación también lo llevas tú: `epsilon_sarsa` y `epsilon_q_learning` deciden el valor de ϵ de cada episodio. Diseña el de Sarsa para que el agente aprenda la política óptima determinista, que es con la que se le evalúa.

Durante el entrenamiento, `main.py` evalúa cada 100 episodios la política greedy en 100 posiciones fijas de validación e imprime la tasa de éxito y los pasos medios de los episodios que llegan. Con `--graficas` dibuja las curvas de aprendizaje, con `--mapa` el valor $\max_a \hat{q}(s, a, \mathbf{w})$ y la política greedy sobre el recinto, y con `--render N` muestra N episodios de la política greedy.

Programa los huecos, entrena los dos agentes y analiza cómo influyen en el aprendizaje de cada algoritmo el diseño del *tile coding*, α y el programa de ϵ. Compara por último los dos algoritmos: qué política aprende cada uno y con cuánta experiencia.

5. Evaluación de los agentes: la clasificación

Como habrás observado en el trabajo previo, el diseño del *tile coding* puede impactar en el aprendizaje del agente. Para que practicando comprendas mejor los equilibrios entre particularización y generalización de la experiencia, vamos a medir el rendimiento de vuestros agentes en una competición entre grupos.

- Cada grupo envía un agente entrenado con Sarsa y otro entrenado con Q-learning, cada uno con **2000 episodios como máximo**, que es el valor por defecto de `--episodios`. Se guardan así (para el grupo 7):

  ```
  python main.py --algoritmo ambos --alpha <alpha> --guarda G7 --grupo G7
  ```

  que escribe `G7_sarsa.npz` y `G7_q_learning.npz` con los pesos, el diseño del *tile coding*, los hiperparámetros y la semilla (`--seed`). Antes de enviarlos, compruébalos con

  ```
  python evalua_agente.py G7_sarsa.npz
  ```

  que los carga y los evalúa exactamente como lo haremos nosotros, pero en las posiciones de validación.

- El envío se hace en la carpeta de tu grupo en el Teams de la clase. Contiene los dos ficheros `.npz` y tus cuatro ficheros con huecos: `agente.py`, `recompensa.py`, `sarsa.py` y `q_learning.py`.

- Los profesores evaluaremos los agentes una sola vez, en un banco de 1000 posiciones de partida que no conoceréis, con su política greedy y un máximo de 200 pasos por episodio.

- Habrá dos clasificaciones, y en cada una el grupo puntúa con su mejor agente:

  1. número de episodios que llegan a la zona objetivo;
  2. pasos de más respecto al camino mínimo desde cada posición de partida, de media en los episodios que llegan. Los episodios que chocan o no llegan no cuentan, y solo puntúan los agentes que llegan desde al menos 500 de las 1000 posiciones.

- Volveremos a entrenar cada agente con vuestro código, vuestros hiperparámetros y vuestra semilla, y **un agente cuyos pesos no se reproduzcan exactamente queda fuera de la clasificación**. El entrenamiento es reproducible: toda la aleatoriedad sale de la semilla. Por eso no hay que modificar los ficheros sin huecos, ni tocar el código después de guardar los pesos.

- El grupo que gane la clasificación de llegadas elegirá en primer lugar su turno de presentación en la sesión final del laboratorio, y el que gane la de pasos elegirá en segundo lugar. Si un mismo grupo gana las dos, el segundo turno pasa al siguiente de la clasificación de pasos.

6. Memoria de trabajo

Aunque este trabajo lo vamos a evaluar en la presentación final, debes recoger en un cuaderno de trabajo todo lo que vayas haciendo, incluyendo el trabajo previo. Eventualmente podemos pedírtelo para comprobar alguno de los resultados/desarrollos tras la presentación final.